

# sop-sockets-helper: quick reference

Cheatsheet for socket + epoll labs

## Contents

### 1 What this is

`sop-sockets-helper.c` (with `sop-sockets-helper.h`) is a grab-bag of small, mostly boring wrappers around things you repeatedly type in OS labs: stream sockets (UNIX or TCP), `epoll_ctl`, non-blocking I/O, line buffering, and a few string helpers. It is **not** a drop-in replacement for your course's `l7-common.h` or autograder expectations—you still wire the lab logic yourself.

**Compile tip:** the `.c` file starts with `#define _GNU_SOURCE` so glibc exposes things like `accept4`-friendly patterns and `struct sigaction`. Build with something like:

```
cc -std=c17 -Wall -Wextra -o lab sop-sockets-helper.c your_solution.c
```

### 2 Conventions

- Most functions return 0 on success and -1 on failure, setting `errno` when something went wrong at the syscall level (unless noted).
- `skh_die` / `skh_perror_die` are for *your* debugging convenience; course servers often want graceful handling instead of instant `exit`.
- `TEMP_FAILURE_RETRY` (inside the `.c`) retries syscalls interrupted by `EINTR`.

### 3 Macros and constants

#### 3.1 `SKH_ERRBUF`

Defined in the header as 256. Reserved for future helpers that format human-readable errors (currently unused).

### 4 Fatal / diagnostic helpers

#### 4.1 `void skh_die(const char *ctx, const char *why)`

Prints `ctx` and `why` to `stderr`, then calls `exit(EXIT_FAILURE)`. Use when you want a loud failure during bring-up.

#### 4.2 `void skh_perror_die(const char *ctx)`

Calls `perror(ctx)` (uses `errno`) and exits.

#### 4.3 `void skh_warn(const char *ctx, const char *msg)`

Prints a warning to `stderr` and returns—does not exit.

#### 4.4 `int skh_errno_is_wouldblock(int e)`

Returns true if `e` is `EAGAIN` or `EWOULDBLOCK`. Handy after `read/write/accept` on non-blocking fds.

#### 4.5 `void skh_ignore_sigpipe(void)`

Installs `SIG_IGN` for `SIGPIPE`. Almost always call this once in `main` before you `write` to sockets you do not fully trust.

### 5 Per-fd flags

#### 5.1 `int skh_set_nonblocking(int fd, int on)`

Gets `F_GETFL`, toggles `O_NONBLOCK`, sets with `F_SETFL`. `on` nonzero enables non-blocking mode.

#### 5.2 `int skh_set_cloexec(int fd, int on)`

Gets `F_GETFD`, toggles `FD_CLOEXEC`, sets with `F_SETFD`. Prevents accidental fd leaks across `exec`.

### 6 Filesystem helper

#### 6.1 `int skh_unlink_if_exists(const char *path)`

Calls `unlink(path)`. Treats `ENOENT` as success (nothing to remove). Other errors propagate as `-1`.

### 7 TCP (IPv4)

#### 7.1 `int skh_tcp_listen(uint16_t port, int backlog)`

Creates `AF_INET SOCK_STREAM`, sets `SO_REUSEADDR`, binds `INADDR_ANY`, `listen`. Returns listening socket fd or `-1`.

#### 7.2 `int skh_tcp_connect_ipv4(const char *host, uint16_t port)`

Uses `getaddrinfo` with `AF_INET` hints, walks the result list, tries `socket+connect` until one succeeds. Returns connected fd or `-1`. Sets `errno` to `EINVAL` if name resolution fails in a coarse way.

### 8 UNIX domain stream sockets

#### 8.1 `int skh_unix_stream_socket(void)`

Wrapper for `socket(AF_UNIX, SOCK_STREAM, 0)`.

#### 8.2 `int skh_unix_bind_listen(const char *path, int backlog)`

Unlinks stale socket path, fills `sockaddr_un.sun_path`, `bind`, `listen`. Returns listening fd or `-1`. Fails with `ENAMETOOLONG` if the path does not fit in `sun_path`.

#### 8.3 `int skh_unix_connect(const char *path)`

Client-side `connect` to a UNIX stream address.

## 9 epoll wrappers

### 9.1 `int skh_epoll_create(void)`

`epoll_create1(EPoll_CLOEXEC).`

### 9.2 `int skh_epoll_add(int epfd, int fd, uint32_t events, int u32_or_fd)`

`EPOLL_CTL_ADD`. Stores `u32_or_fd` in `event.data.fd` (the usual “remember which fd this is” trick).

### 9.3 `int skh_epoll_add_ptr(int epfd, int fd, uint32_t events, void *ptr)`

Same but `event.data.ptr = ptr` for when you want a pointer to your `client_t`.

### 9.4 `int skh_epoll_mod(...)` / `skh_epoll_mod_ptr(...)`

`EPOLL_CTL_MOD` variants mirroring the add pair.

### 9.5 `int skh_epoll_del(int epfd, int fd)`

`EPOLL_CTL_DEL` with `NULL` event (Linux allows this).

## 10 Raw read / write

### 10.1 `ssize_t skh_read_some(int fd, void *buf, size_t cap)`

Single read with `EINTR` retry. Returns byte count, 0 on EOF, -1 on error (including `EAGAIN` on non-blocking fds—caller decides).

### 10.2 `ssize_t skh_write_all(int fd, const void *buf, size_t len)`

Loops `write` until all bytes are sent or an error occurs. If `write` returns 0, sets `errno` to `EPIPE` and returns -1. **Not** suitable alone for non-blocking sockets without handling `EAGAIN` (you would need an output queue or `EPOLLOUT`).

## 11 Growable line buffer: `skh_linebuf`

A simple “append bytes from `read`, then peel off newline-terminated lines” helper. Data is always kept `NUL`-terminated for convenience (`len` does not count the extra guard byte in the same way as a raw byte ring).

### 11.1 `void skh_lb_init(skh_linebuf *b)`

Nulls pointers and sizes.

### 11.2 `void skh_lb_free(skh_linebuf *b)`

Frees data and zeros the struct.

### 11.3 `void skh_lb_clear(skh_linebuf *b)`

Sets logical length to 0 without freeing capacity.

### 11.4 `int skh_lb_reserve(skh_linebuf *b, size_t need)`

Ensures `cap >= need` by doubling growth (starts at 128). Returns -1 on allocation failure.

**11.5** `ssize_t skh_lb_read_append(skh_linebuf *b, int fd, size_t max_append)`

Reserves space, reads up to `max_append` bytes at the end of the buffer, updates `len`, NUL-terminates. Returns read result (bytes read) or `-1`.

**11.6** `char *skh_lb_extract_line(skh_linebuf *b, size_t max_line_including_nul)`

If a `'\n'` exists in the buffer, allocates a `malloc`'d string containing the line **including** the newline character, removes that prefix from the internal buffer, and returns the allocation. Returns `NULL` if no full line yet. If the line would exceed your `max_line_including_nul` budget (allocator size), sets `errno` to `EMSGSIZE` and returns `NULL`.

**11.7** `void skh_lb_drop_prefix(skh_linebuf *b, size_t n)`

memmoves away the first `n` bytes (used if you parse manually).

## **12 FIFO byte chunk: `skh_ringbuf`**

Despite the name, this is a **linear** buffer with indices `head` and `tail`: used region is `[head, tail)`. When `tail` hits `cap`, bytes are compacted to the front. Good enough for labs; not a lock-free SPSC ring.

**12.1** `void skh_rb_init(skh_ringbuf *r, size_t cap)`

Allocates at least `cap` bytes (default 4096 if `cap==0`). If `malloc` fails, `cap` becomes 0.

**12.2** `void skh_rb_free(skh_ringbuf *r)`

Frees storage.

**12.3** `size_t skh_rb_used(const skh_ringbuf *r)`

Returns `tail - head`.

**12.4** `size_t skh_rb_free_cap(const skh_ringbuf *r)`

Returns contiguous free space at the end (`cap - tail`).

**12.5** `ssize_t skh_rb_read_fd(skh_ringbuf *r, int fd)`

Compact if full, grow if needed, read into tail region, advance `tail`.

**12.6** `int skh_rb_find_byte(const skh_ringbuf *r, unsigned char c, size_t *out_pos)`

Linear search for a byte (e.g. `'\n'`). Sets `*out_pos` to index if found.

**12.7** `void skh_rb_pop(skh_ringbuf *r, size_t n)`

Advances `head` by `n` (drops consumed prefix).

**12.8** `ssize_t skh_rb_write_fd(skh_ringbuf *r, int fd)`

Writes the entire used region with `skh_write_all`, then resets `head` and `tail` to 0 on success.

## 13 Accept helper

### 13.1 `int skh_accept_nonblocking(int listenfd)`

`accept` with `EINTR` retry, then forces non-blocking mode and `FD_CLOEXEC` on the new fd. Returns client fd or `-1`.

## 14 Small string utilities

### 14.1 `char *skh_strdup_line(const char *start, size_t len)`

`malloc(len+1)`, copies bytes, NUL-terminates. Caller frees.

### 14.2 `void skh_trim_crlf_inplace(char *s)`

Strips trailing `'\r'` and `'\n'` characters in place.

### 14.3 `char *skh_split_first_colon(char *line, char **msg_out)`

Finds the first `':'`, writes `'\0'` there, sets `*msg_out` to the character after the colon, returns `line` (now the “left side”). Returns `NULL` if no colon. **Mutates** the buffer.

### 14.4 `int skh_parse_u16(const char *s, uint16_t *out)`

Strict unsigned decimal parse into `uint16_t` (whole string must match).

### 14.5 `int skh_parse_positive_int(const char *s, int *out)`

Strict parse of a positive `int` (must be  $\geq 1$ ).

### 14.6 `size_t skh_strlcpy(char *dst, const char *src, size_t dstsz)`

BSD-style bounded copy: always NUL-terminates if `dstsz > 0`. Returns `strlen(src)` (length of the *source* string), which may be  $\geq \text{dstsz}-1$  if truncated.

## 15 How to actually use this tomorrow

1. Skim the task: UNIX or TCP? line protocol or raw bytes? `epoll` edge or level? `stdin`?
2. Pick helpers: `skh_unix_bind_listen` vs `skh_tcp_listen`; `skh_lb_*` vs manual `read` loop.
3. Glue your state machine (`client_t` array, partner links, timeouts) **yourself**—the helpers do not know your story.
4. If the grader forbids extra files, you can copy-paste individual functions (keep license/credits honest with course rules).